
LAPORAN PRAKTIKUM

Internet of Think



Oleh:

Nama : Azzril Andrean

NPM : 25782034

Kelas : TRI 3B

Dosen

Agus Ambarwari, S.Pd., M.kom

Dr. Ir. Septafiansyah Dwi Putra

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026/2027

Modul 4: Komunikasi Asinkron dan WebSockets

4.1 Pengantar

Pada Modul 3, kita telah berhasil membangun jembatan interaksi antara dunia fisik dan antarmuka web. Namun, metode HTTP tradisional memiliki batasan arsitektur, di mana *Web Server* akan berhenti sejenak saat memproses beban kerja (*blocking*), dan data di layar *browser* tidak akan diperbarui kecuali pengguna me-*refresh* halamannya secara manual atau sistematis (*flickering*). Memasuki Modul 4, kita akan mempelajari arsitektur yang lazim digunakan di sistem modern seperti *Smart Home Dashboard* komersial, yaitu WebSockets. Dengan WebSockets, layar ponsel Anda dapat bereaksi seketika dalam hitungan milidetik saat sakelar fisik di dinding ditekan, tanpa ada *loading* pada halaman web sama sekali.

4.1 Instalasi Pustaka Khusus (*Manual via GitHub*)

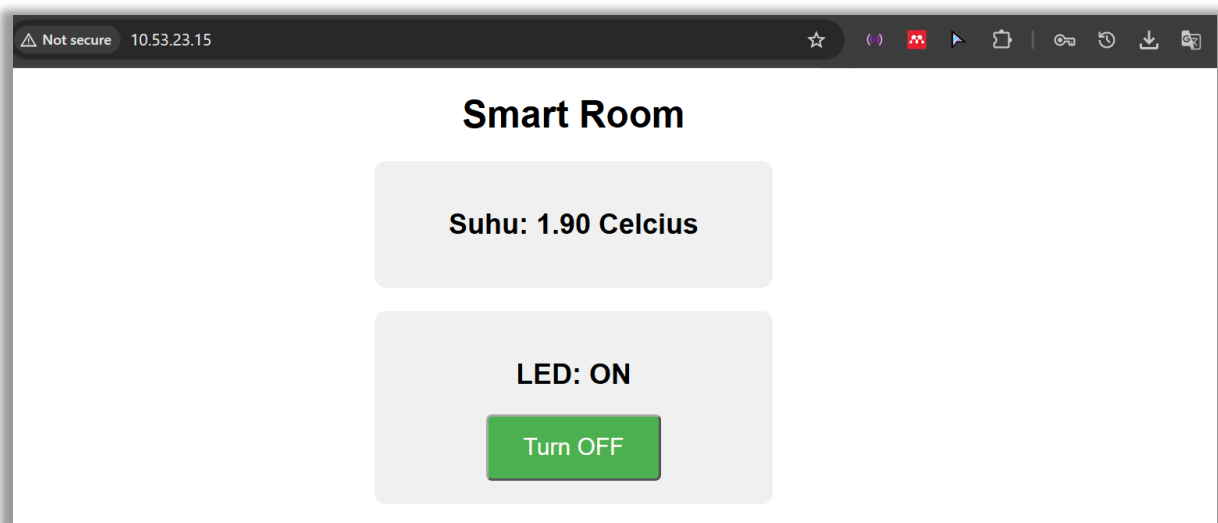
Karena berbasis teknologi Asinkron pihak ketiga, Anda harus mengunduh kode sumber pustaka ini secara manual dalam format .zip dari GitHub, lalu memasangnya ke Arduino IDE via menu **Sketch > Include Library > Add .ZIP Library....**

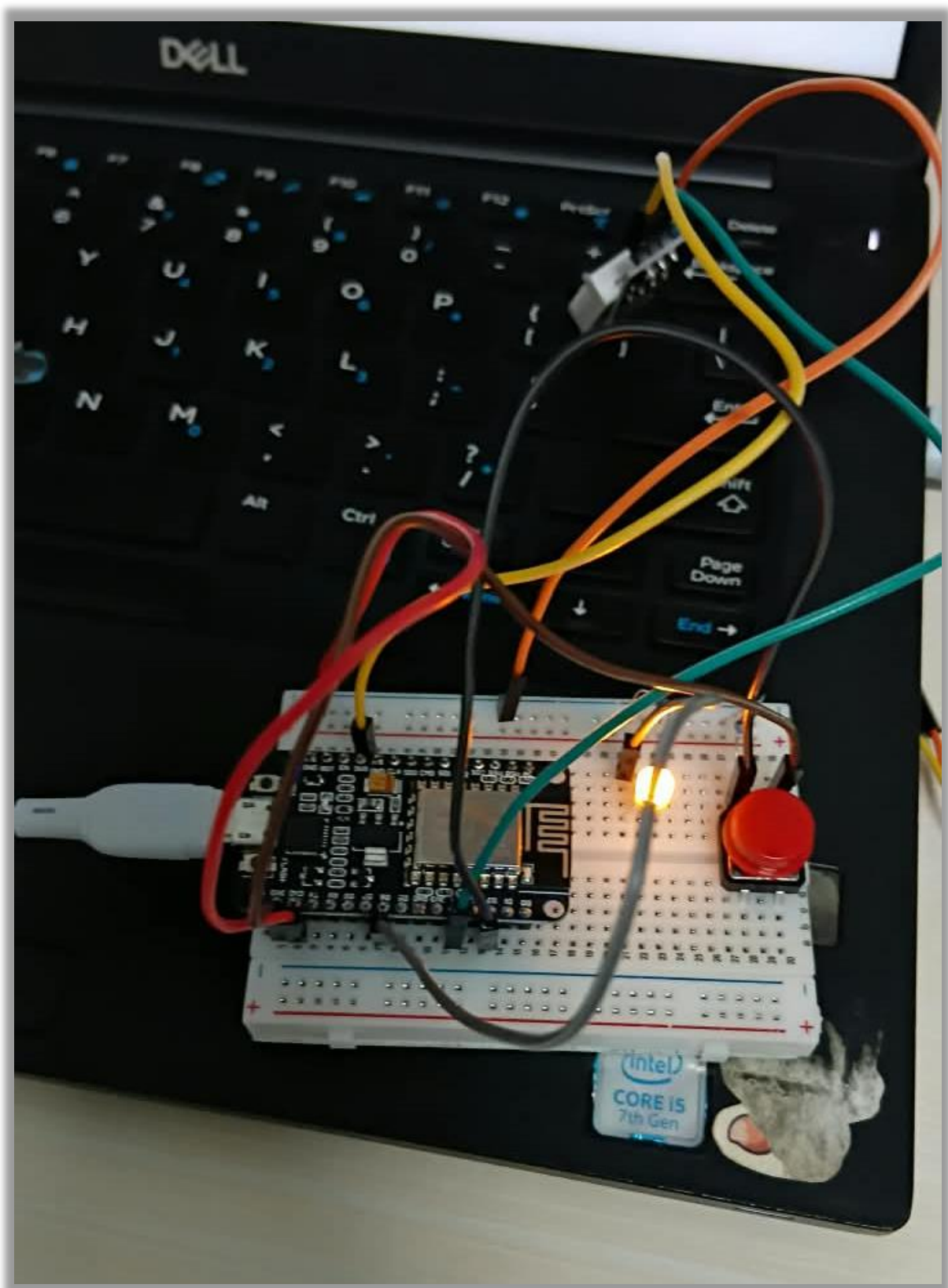
Silakan kunjungi tautan berikut dan unduh berkas ZIP dari menu Code > Download ZIP:

1. ESPAsyncWebServer: <https://github.com/ESP32Async/ESPAsyncWebServer>
2. ESPAsyncTCP: <https://github.com/ESP32Async/ESPAsyncTCP>

(Perhatian: Jika Anda menggunakan mikrokontroler seri ESP32 di luar praktikum ini, pustaka TCP-nya bernama *AsyncTCP*).

4.2 Praktikum 1: Real-Time Dashboard & Sinkronisasi Tombol Fisik





4.3 Latihan

Kerjakan latihan berikut untuk menguji pemahaman Anda terhadap arsitektur web modern!

- Latihan 1 (Teori Jaringan): Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

Jawab : WebSocket jauh lebih hemat bandwidth karena koneksinya dibuka sekali saja lalu dibiarkan tetap terbuka (persistent), sehingga data dapat dikirim dua arah kapan saja tanpa harus membuat koneksi baru. Sebaliknya, Meta Refresh HTTP memaksa browser mengirim request baru secara berkala (misalnya tiap beberapa detik), dan setiap request harus melalui proses handshake TCP serta membawa header HTTP yang besar.

- Latihan 2 (Analisis Waktu/Timer): Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks `delay(3000)` di dalam siklus fungsi `void loop()` pada draf arsitektur kompleks ini?

Jawab : Memanggil `delay(3000)` di dalam `loop()` akan memblokir CPU selama 3 detik di setiap siklus, sehingga semua tugas lain berhenti: tombol fisik tidak terbaca, perintah `toggle` dari browser dan respons LED jadi lambat, dan `ws.cleanupClients()` jarang dipanggil. Koneksi WebSocket juga bisa tertunda atau timeout lalu terputus, dan jika ada proses berat lain, watchdog timer dapat memicu reset otomatis pada board. Karena itu digunakan pola non-blocking dengan `millis()`, yaitu hanya mengecek selisih waktu tanpa menghentikan program, sehingga tombol, WebSocket, dan pembacaan DHT berjalan hampir bersamaan secara real-time.

- Latihan 3 (Modifikasi Front-End): Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML (`<span id="...">`) apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

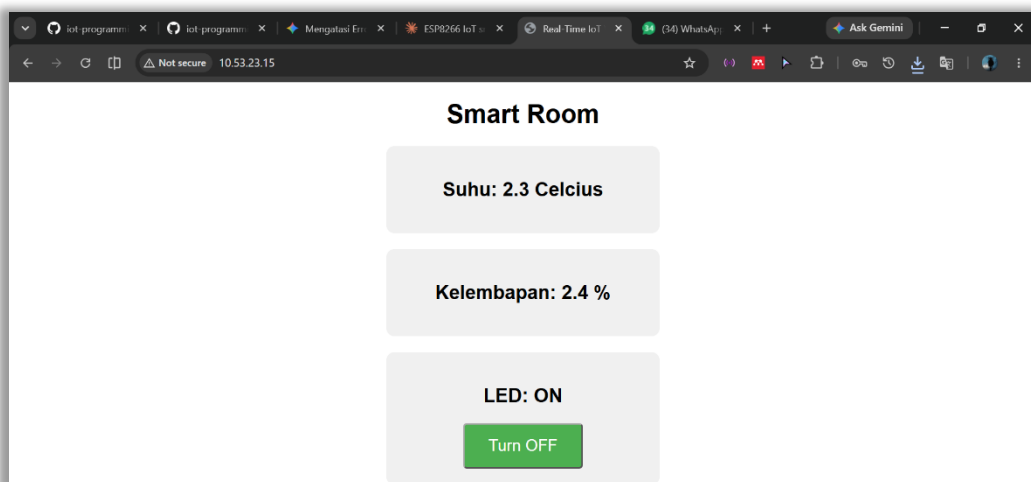
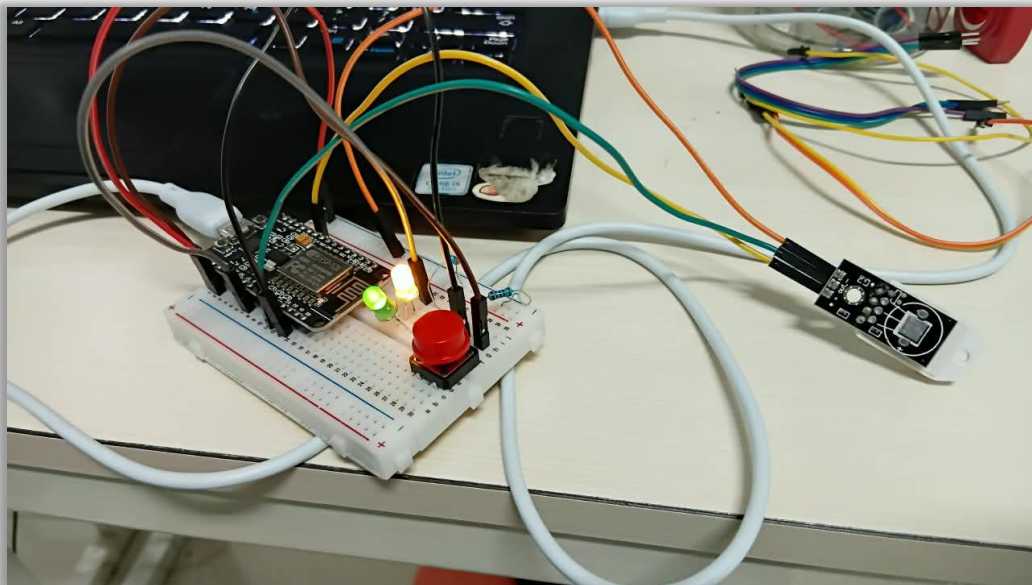
Jawab : Pada blok // Merombak UI Tombol & Teks sesuai status hardware, script menargetkan dua ID elemen lewat `document.getElementById()`, yaitu `ledStatus` (`<span id="ledStatus">`) yang teksnya diubah menjadi "ON" atau "OFF" sesuai nilai `dataObj.led`, dan `toggleBtn` (`<button id="toggleBtn">`) yang teks dan kelas CSS-nya diubah menjadi "Turn OFF"/`btn-on` (hijau) atau "Turn ON"/`btn-off` (merah). Jadi, khusus elemen `<span>`, targetnya hanya `ledStatus`; sementara `tempValue` juga berupa `<span>`, tetapi diubah pada blok sebelumnya (**// Menyuntikkan teks angka Suhu...**) untuk menampilkan suhu, sehingga secara keseluruhan fungsi `onMessage` menginjeksi tiga ID: `tempValue`, `ledStatus`, dan `toggleBtn`.

- Latihan 4 (Pengembangan Telemetry Sensor - Kelembapan): Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program

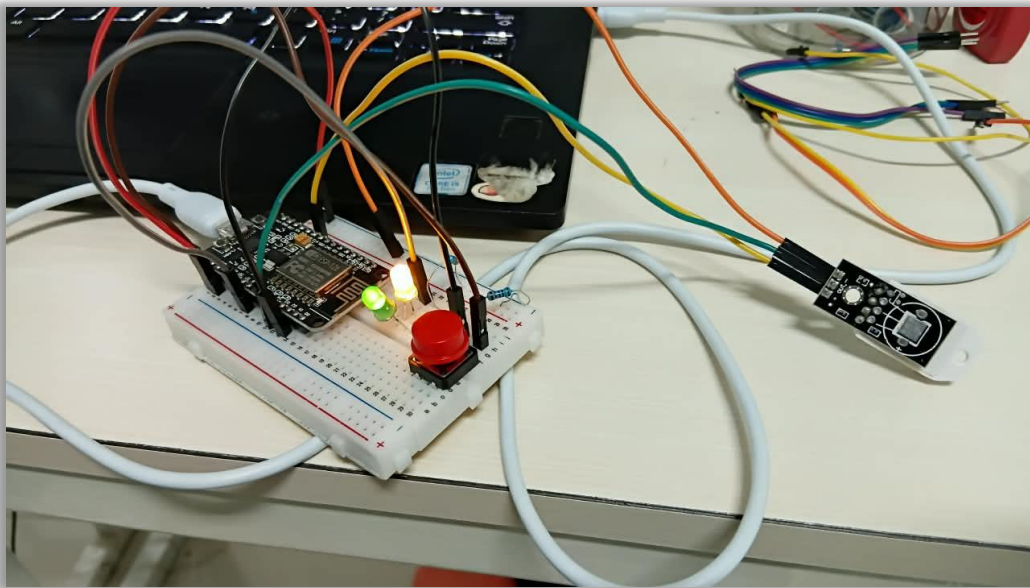
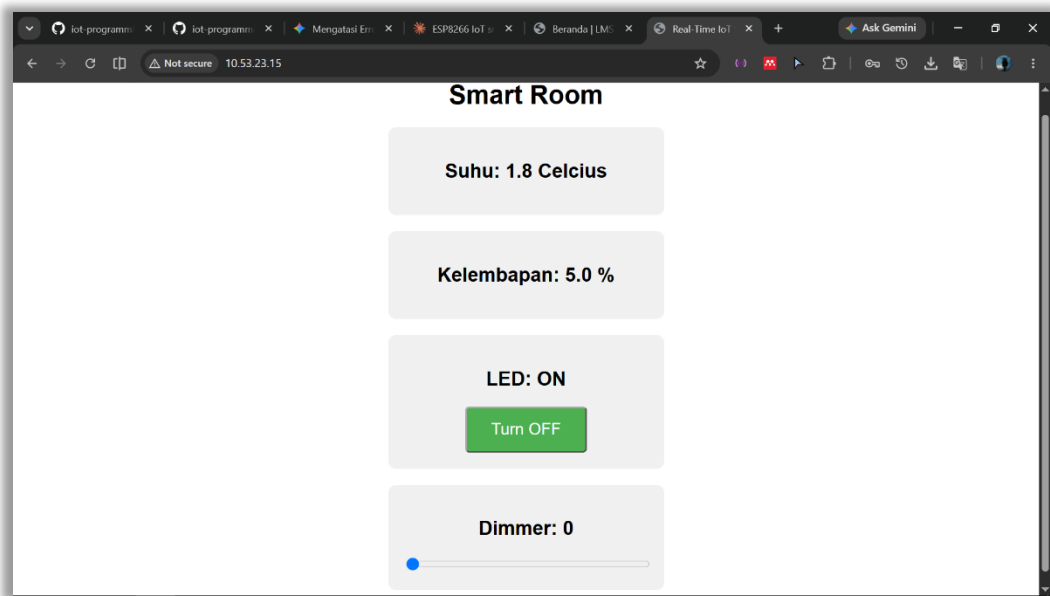
agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via WebSockets:

- i. Sisi Back-End (C++): Buat variabel penampung (misal: String currentHum = "-");. Baca nilai kelembapan menggunakan fungsi `dht.readHumidity()` di dalam siklus timer *non-blocking* `millis()`. Perbarui fungsi `notifyClients()` agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke *browser* (contoh format: `{"led":"...", "suhu":"...", "hum":"..."}`).
- ii. Sisi Front-End (HTML & JavaScript): Tambahkan elemen kartu (`<div class="card">`) baru pada kerangka HTML sebagai penampil kelembapan (misal: `<h2>Kelembapan: <span id="humValue">--</span> %</h2>`). Kemudian, modifikasi fungsi JavaScript `onMessage(event)` agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen `humValue`.

Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!



4.4 Tugas: Sistem Kendali Intensitas Cahaya (PWM Slider) via *WebSockets*



LAPORAN PRAKTIKUM

Internet of Think



Oleh:

Nama : Azzril Andrean

NPM : 25782034

Kelas : TRI 3B

Dosen

Agus Ambarwari, S.Pd., M.kom

Dr. Ir. Septafiansyah Dwi Putra

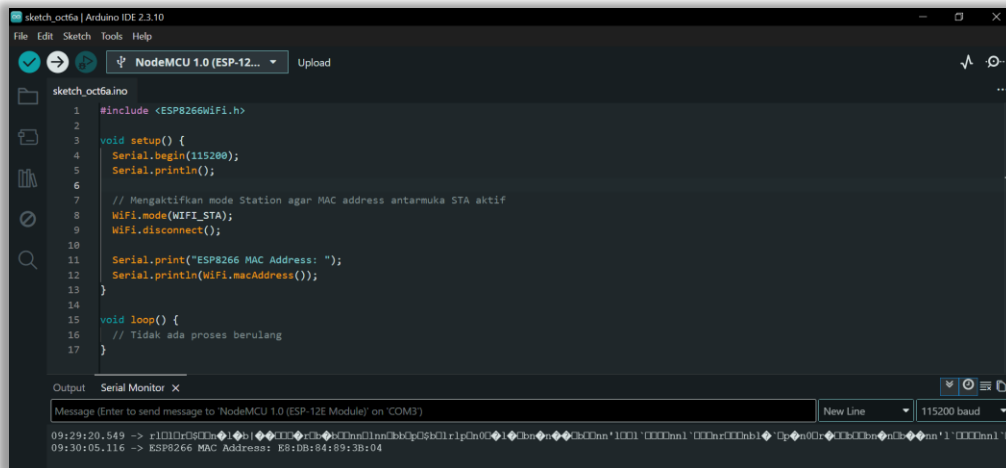
TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

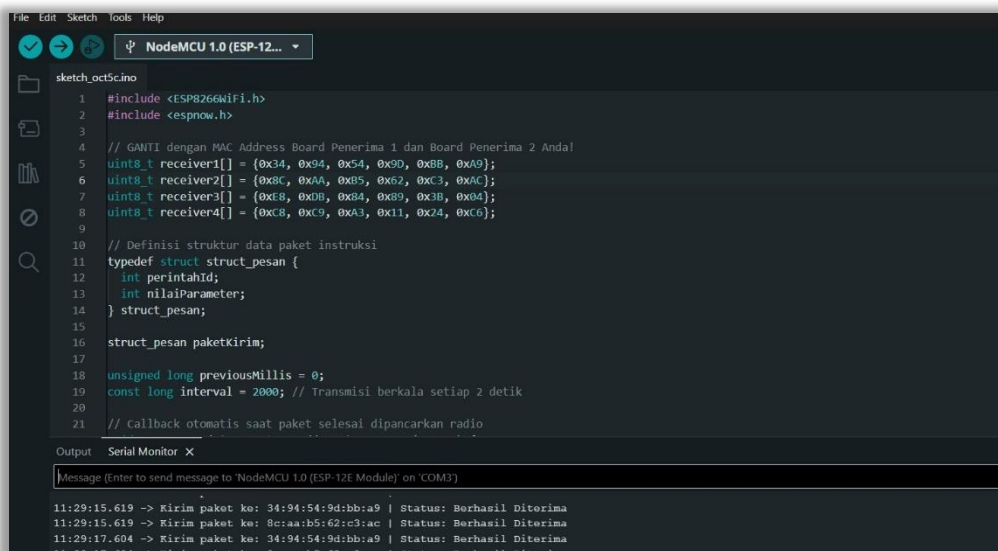
2026/2027

Praktikum 1: Identifikasi MAC Address Perangkat NodeMCU

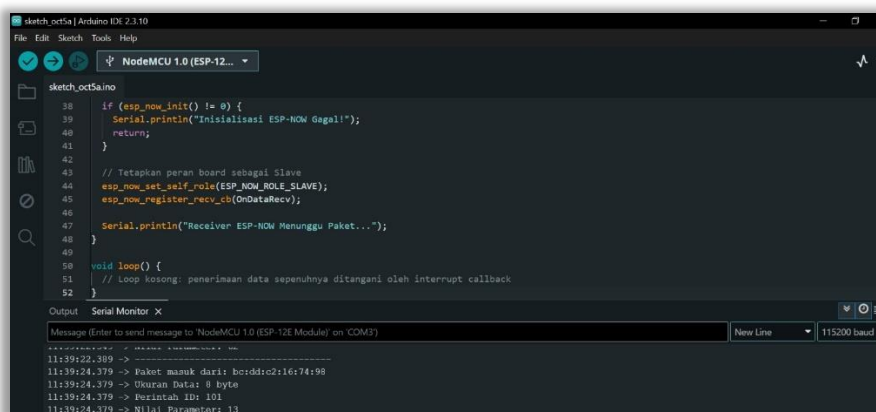


Menampilkan Mac Address dari perangkat esp kita

Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW)

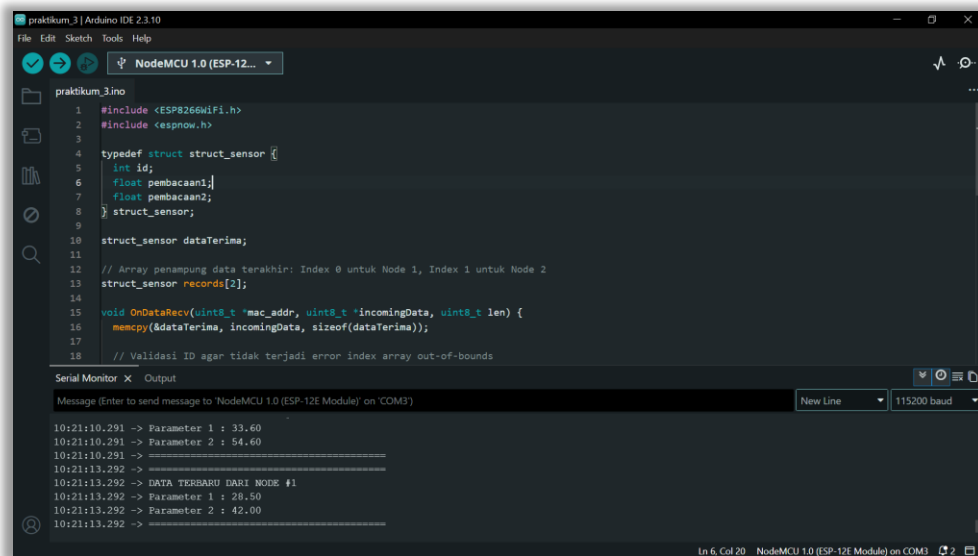


Sisi Penerima (Slave):



Praktikum 3: Pengumpulan Data Telemetri Many-to-One (ESP-NOW)

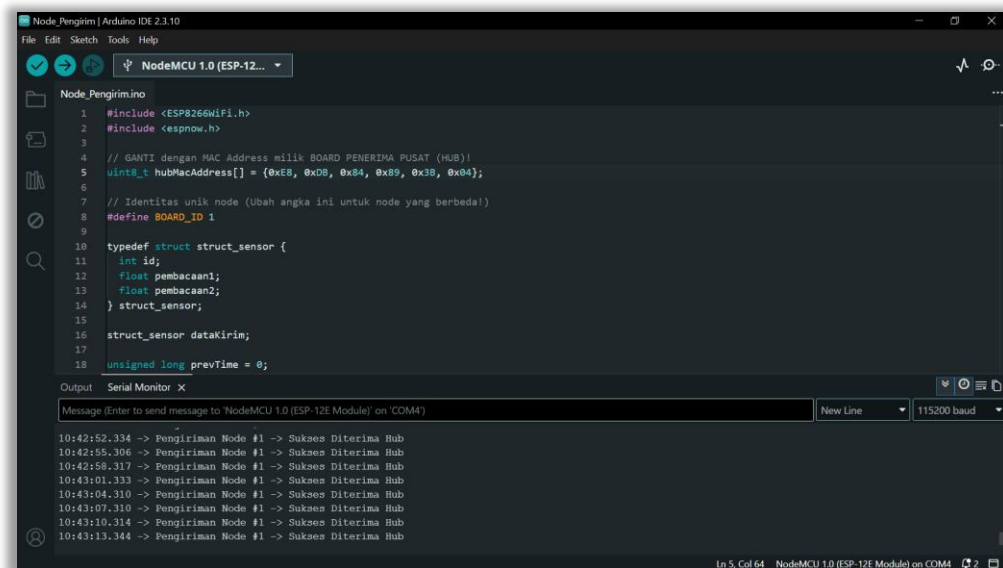
Sisi Hub Konsentrato



The screenshot shows the Arduino IDE interface with the file 'praktikum_3.ino' open. The code is for a NodeMCU 1.0 (ESP-12E) module. It includes `<ESP8266WiFi.h>` and `<espnw.h>`. It defines a `struct_sensor` type with fields `int id;`, `float pembacaan1;`, and `float pembacaan2;`. A `struct_sensor dataTerima;` is declared. An array `struct_sensor records[2];` is defined for storing data from two nodes. The `OnDataRecv` function is implemented to receive data from a specific MAC address and store it in the `records` array. The serial monitor shows the following output:

```
10:21:10.291 -> Parameter 1 : 33.60
10:21:10.291 -> Parameter 2 : 54.60
10:21:10.291 -> =====
10:21:13.292 -> =====
10:21:13.292 -> DATA TERBARU DARI NODE #1
10:21:13.292 -> Parameter 1 : 28.50
10:21:13.292 -> Parameter 2 : 42.00
10:21:13.292 -> =====
```

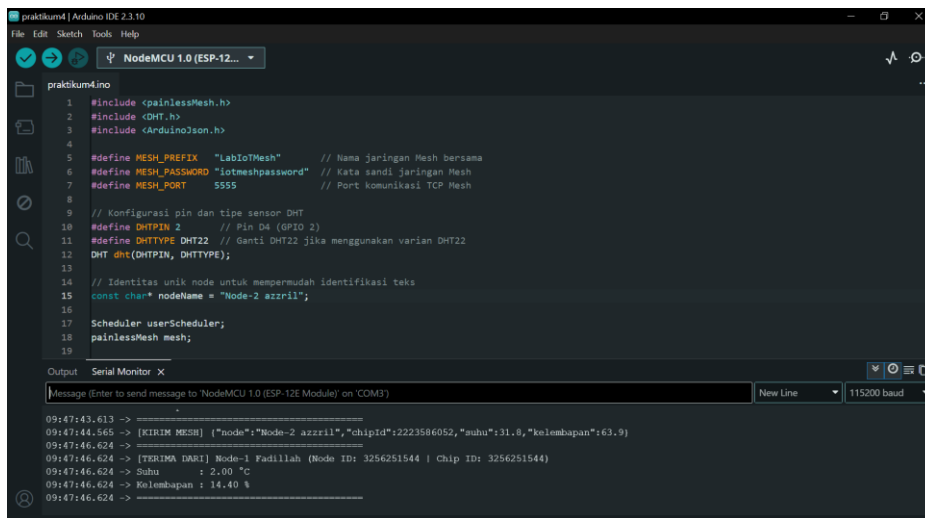
Sisi Node Pengirim:



The screenshot shows the Arduino IDE interface with the file 'Node_Pengirim.ino' open. The code is for a NodeMCU 1.0 (ESP-12E) module. It includes `<ESP8266WiFi.h>` and `<espnw.h>`. It defines a `uint8_t hubMacAddress[] = {0xE8, 0xD8, 0x84, 0x89, 0x3B, 0x04};` and a `#define BOARD_ID 1`. It defines a `struct_sensor` type with fields `int id;`, `float pembacaan1;`, and `float pembacaan2;`. A `struct_sensor dataKirim;` is declared. The `prevTime` variable is set to 0. The serial monitor shows the following output:

```
10:42:52.334 -> Pengiriman Node #1 -> Sukses Diterima Hub
10:42:55.306 -> Pengiriman Node #1 -> Sukses Diterima Hub
10:42:58.317 -> Pengiriman Node #1 -> Sukses Diterima Hub
10:43:01.333 -> Pengiriman Node #1 -> Sukses Diterima Hub
10:43:04.310 -> Pengiriman Node #1 -> Sukses Diterima Hub
10:43:07.310 -> Pengiriman Node #1 -> Sukses Diterima Hub
10:43:10.314 -> Pengiriman Node #1 -> Sukses Diterima Hub
10:43:13.344 -> Pengiriman Node #1 -> Sukses Diterima Hub
```

Praktikum 4: Jaringan Sensor Ad-Hoc Multi-Hop (painlessMesh)



The screenshot displays the Arduino IDE interface for a project named 'praktikum4'. The code is for a NodeMCU 1.0 (ESP-12E) and implements a painlessMesh network. The code includes headers for painlessMesh, DHT, and ArduinoJson. It defines a mesh prefix 'LabIoTMesh', a password 'iotmeshpassword', and a port '5555'. It configures a DHT sensor on pin 2. A unique node name 'Node-2 azzril' is defined. The main setup includes a scheduler and the painlessMesh mesh object.

```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "LabIoTMesh" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "iotmeshpassword" // Kata sandi jaringan Mesh
7 #define MESH_PORT 5555 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-2 azzril";
16
17 Scheduler userScheduler;
18 painlessMesh mesh;
19
```

The Serial Monitor shows the following output:

```
09:47:43.613 -> =====
09:47:44.565 -> [KIRIM MESH] {"node":"Node-2 azzril","chipid":2223586052,"suhu":31.8,"kelembapan":63.9}
09:47:46.624 -> =====
09:47:46.624 -> [TERIMA DARI] Node-1 Fadillah (Node ID: 3256251544 | Chip ID: 3256251544)
09:47:46.624 -> Suhu : 2.00 °C
09:47:46.624 -> Kelembapan : 14.40 %
09:47:46.624 -> =====
```